

Concurrent Operations in a Distributed and Mobile Collaborative Environment

Maher Suleiman, Michèle Cart, Jean Ferrié

LSI, University of Montpellier
860, rue de St Priest, 34090 Montpellier, France
e-mail: {suleiman, cart, ferrie}@lsi.fr

Abstract

In a distributed groupware system, objects shared by users are subject to concurrency and real-time constraints. In order to satisfy these, various concurrency control algorithms [4] [10] [18] have been proposed that exploit the semantic properties of operations. By ordering concurrent operations, they generally guarantee consistency of the different copies of each object. However, in some situations they can result in inconsistent copies, a non-respect of user's intentions, and in the need to undo and redo some operations.

The principle objective of this paper is to present an algorithm that overcomes these drawbacks in distributed and mobile collaborative environments. The algorithm is based on the notion of user's intention, and also on the construction of equivalent histories by exploiting and combining some general semantic properties such as forward/backward transposition.

1 Introduction

The objective of a collaborative environment is to facilitate team working and, in particular, to enable a group of persons to manipulate shared objects, and modify them in a coherent manner. There are many collaborative activities, examples being simultaneous writing of a document by different authors, or cooperative design. In general, an object involved in a collaborative activity is submitted to concurrent accesses and real-time constraints. The real-time aspect necessitates every user seeing the effects of his own acts on the object immediately, and the effects of the acts of other users almost immediately. Consequently, the problem is to conciliate both real-time and consistency constraints, as the object may be modified concurrently by many users. To satisfy these requirements, it is necessary [5] [8] that concurrency control does not use a blocking protocol which could defer user actions.

Some collaborative systems [9] [3] [15] use locks for concurrency control, thus preventing simultaneous access to the object. Other systems [10] [4] [18] use multiple copies to permit concurrent access to the same object, and use semantic properties of the operations to reorder concurrent operations while preserving object consistency. More

precisely, the works described in [4] use a transformation property to construct the history associated with each copy of the object. The histories of the different copies of the object are guaranteed to be equivalent (i.e. lead to the same final state), although the order of concurrent operations can be different. However, the problem of partial concurrency evoked in [1], that can lead to incoherence, is not solved. The works described in [10] use commutativity and masking properties to reorder the histories associated with object copies so as to respect a unique order defined *a priori* by timestamps. Unfortunately, in the absence of commutativity and masking properties, some operations have to be undone and then redone. In addition, the user intention is not always respected. In [18], the transformation property is used in order to respect the user intention. Moreover, the histories of the different copies of the object are ordered by using the same order to guarantee consistency of the object. But unfortunately, as in [4], the problem of partial concurrency is not solved and as in [10], operations have to be undone and then redone in order to respect the same order.

In this paper we aim to design a system that does not have these drawbacks. Our objective is to satisfy both concurrency and real-time constraints, while solving the partial concurrency problem and respecting the user intention without undoing and redoing operations.

The paper is organised into six sections. Section 2 presents the model and the problematic for a distributed collaborative environment. The three following sections detail the principles of the solution, based on the construction of equivalent histories. Section 3 recalls the technique used for respecting the causality. Section 4 presents the notion of intention and the use of transpositions for respecting the intentions of users. Section 5 expresses conditions that are sufficient to guarantee the consistency of the copies. Algorithms that use forward as well as backward transposition and causal broadcast are presented in Section 6. Finally, the last section extends the model for supporting mobile sites.

2 Model and Problematic

A distributed collaborative system is constituted of a set of sites interconnected by a reliable network and of set of

objects (i.e. texts, graphics, ...) shared by the users collaborating in a common activity. We suppose as well, to simplify the presentation, that there is one user per site. The shared objects are distributed and replicated. Thus, there exists, for each object, a copy at every site. Each object O may be manipulated by users by means of specific operations $\{op_1, op_2, \dots, op_n\}$. In order to maintain the consistency of the copies of O , every operation intended to be executed on O must be executed on all the copies. For that, an operation generated by a site must be broadcast to all the other sites. The progress of an operation is decomposed into four steps: *generation*, *broadcast*, *reception*, and *execution*. Thus, an operation *generated* by a site is *sent* to all sites (including the generator site itself) where it is *received* and then *executed*.

In order to guarantee users a minimum response time, the execution of an operation on the generator site must not be delayed by the mechanism ensuring the consistency of the copies. More precisely, an operation generated by a site must be executed immediately after its reception by this site. In the following, for a site S , a *local* operation is an operation generated by S , while a *remote* operation designates an operation generated by a site other than S . Let us specify the causality relation that may exist between two operations.

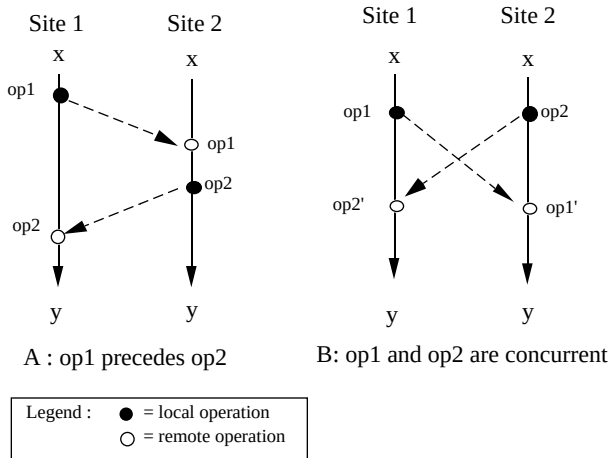


Figure 1. Concurrency / precedence relations between operations.

Definition 1. Causal precedence between two operations.

We say that op_1 *precedes* op_2 iff op_2 has been generated by site S after the execution of op_1 on S (Fig. 1A).

Definition 2. Concurrency between two operations.

We say that op_1 and op_2 are *concurrent* (Fig. 1B) iff:

not (op_1 *precedes* op_2) and not (op_2 *precedes* op_1).

The *precede* relation expresses the potential causality [16] between these operations. If op_1 precedes op_2 , op_2 is supposed to be dependent on the effects produced by op_1 . On the contrary, if op_1 and op_2 are concurrent it means that these operations are completely independent and that neither one has seen the effects of the other. The relation *precede* must be respected on all sites; in other words, if op_1 precedes op_2 , then op_1 must be executed before op_2 on all sites. If op_1 and op_2 are concurrent, they may be executed in any order on different sites. However, as the first operation

changes the object's state, the second operation cannot be executed on the new object's state; otherwise, it would give unacceptable results because they would be different from what the user was expecting. For that reason, we will see in section 4 that the operation which is executed second must be transformed to take the effects produced by the first operation into account and to satisfy the user's intention.

In this context, the problem is to integrate on each site the effects of local and remote operations while respecting the following constraints: (i) immediate execution of local operations after receiving them, (ii) respect of the causality, expressed by the relation *precede*, between the operations, (iii) respect of the intentions of users associated to the operations, (iv) guarantee of the consistency of the copies.

The next sections show how to satisfy each of these constraints. For the moment, let us define some notations that will be used later.

Notations

$O_0, O_1, \dots, O_i, O_n, O_f$: designate states of object O .

$O_i.op = O_f$: means that the execution of the operation op has changed object state from O_i to O_f .

$O_0.op_1.op_2 \dots op_n = O_f$: means that the execution, in that order, of the sequence $op_1.op_2 \dots op_n$ has changed object state from O_0 to O_f .

Definition 3. Equivalence of sequences of operations.

Two sequences of operation seq_1 and seq_2 are equivalent (noted $seq_1 \equiv seq_2$) if, starting from the same state, they lead the object to the same state, that is to say:

$$\forall O_i, O_i.seq_1 = O_i.seq_2.$$

Definition 4. Object history on a site. The history of an object O on a site S , noted $H_{S,O}(n)$, is a sequence of n operations executed on the copy of O on the site S that transformed the object O from its initial state to its current state.

So, the problem is to construct, for every site S and for every object O , a history $H_{S,O}$ respecting the constraints above in such a way that if the object copies were identical at the beginning of the work session, they will also be identical each time the system is quiescent [4], that is to say when there are no more messages circulating in the network.

3 Respecting the Causality

To be able to respect the causality between operations we need a technique that allows us to: (1) compare two operations to determine if one operation precedes the other; (2) deduce, when one site receives an operation, if there exists an operation that precedes it and that has not yet been received.

The technique suggested in [4], like that described in [13], satisfies both these criteria. It uses a state vector for each site that counts operations generated by the other sites and executed on this site. We will adopt this technique to define two procedures for message emission and reception. These procedures guarantee a causal broadcast [16].

State Vector. Let N be the number of sites in the collaborative system. These sites are numbered from 1 to N . On every site S , and for every object O , a vector $V_{S,O}$ composed of N elements is defined where: $V_{S,O}[j] = \text{number}$

of operations generated by site j , received by the site S and executed by S on its copy of the object O .

Generation and Broadcast of an Operation. When an operation is generated by a site S on an object O , the current value of the state vector $V_{S,O}$ is associated with it. It is designated by V_{op} . The message broadcast to sites is a triplet: $\langle op, S_{op}, V_{op} \rangle$ where op is the generated operation, S_{op} the generator site and V_{op} the state vector associated with the operation. To simplify the notation in the next sections, we suppose the existence of only one object. Therefore, the notations V_S (resp. H_S) will be used instead of $V_{S,O}$ (resp. $H_{S,O}$). The state vectors associated with operations enable us to compare operations and to determine whether an operation precedes another one.

Reception. When a site S receives an operation op with its state vector V_{op} coming from site S_{op} , this operation must not be executed by the application until all operations that precede it have been received and executed on S . The procedure that undertakes the reception is the following.

procedure receive(op, S_{op}, V_{op}) :
 wait until $V_S[i] \geq V_{op}[i], (\forall i : 1 \leq i \leq N)$;
 execute (op) ;
 $V_S[S_{op}] := V_S[S_{op}] + 1$;

If the operation is generated by site S (it is a local operation), then $S_{op} = S$; in this case, the waiting condition is true and so this local operation is executed immediately. This procedure guarantees a causal reception such as when an operation op is received and executed on a site S , then:

- all the operations which precede op have already been executed on this site,
- op does not precede any operation executed before it on this site.

Thanks to this reception procedure, the following lemmas can be proved. The proofs can be found in [17].

Lemma 1. *Causal precedence between two operations.*

Given two messages $\langle op_1, S_{op1}, V_{op1} \rangle$ and $\langle op_2, S_{op2}, V_{op2} \rangle$, we have:

op_1 precedes op_2 iff $V_{op1}[S_{op1}] < V_{op2}[S_{op1}]$.

Lemma 2. *Concurrent operations.*

Let $\langle op_1, S_{op1}, V_{op1} \rangle$ and $\langle op_2, S_{op2}, V_{op2} \rangle$ be two messages received by site S such that the execution of op_1 on S preceded that of op_2 . Then, op_1 and op_2 are concurrent iff $V_{op1}[S_{op1}] \geq V_{op2}[S_{op1}]$.

4 Respecting the Intentions of the Users

4.1 Intention of the User.

Let us take the example of a text editor. The text is composed of sentences; each sentence is an object represented by a string of characters. Operations defined on this object are:

insert(p, c) : inserts the character c at the position p (before p^{th} character);

delete(p) : delete the p^{th} character.

Consider the object O whose state is the string of characters "system". When a user wants to add the letter 's' at

the end of this word, he puts the cursor after the 'm' and he enters an 's'. The editor translates this action into the operation *insert*(7, 's') which inserts 's' at position 7. We say that the user intention is {add 's' at the end of 'system'} and this intention is realized by the operation *insert*(7, 's').

The intention is an abstraction of the result to be obtained. The operation that realizes it must take the object state into account. So, if the state of the object O were "the system of...", the same intention would be realized by the operation *insert*(11, 's').

In order to distinguish the intention from the operation that realizes it, we will use the following notations:

Operation(I, O_i): designates the operation that realizes the intention I on the object in the state O_i ;

Intention(op, O_i): expresses the user intention realized by op executed on the state O_i .

For example:

Operation({add 's' after 'system'}, "system of") = *insert*(7, 's'),

Intention(*insert*(7, 's'), "system of") = {add 's' after 'system'}.

The abbreviated notation ($O_i.op \rightarrow I$) will be also used to express that the operation op executed on the object in the state O_i , realizes the intention I . This notation is generalized to a sequence of operations; thus, to express that a sequence of operations $[op_1.op_2....op_n]$, executed in that order on O_i , realizes the series of intentions $I_1.I_2....I_n$ we will write: $O_i.op_1.op_2....op_n \rightarrow I_1.I_2....I_n$.

4.2 Serialization of Two Concurrent Operations

The problem is to execute concurrent operations serially while respecting the intentions of the users. To achieve that, a site may be led to transform a received operation, before executing it.

Example 1. Let us take the example of collaborative text editing. Suppose that two users, U_1 on site 1 and U_2 on site 2, work simultaneously on the same object, whose state O is "on systems ...". Suppose that they want to change O into $O' = \text{"one system ..."}$. On site 1, user U_1 puts the cursor after 'on' and then inserts the character 'e'. The operation generated by site 1 is $op_1 = \text{insert}(3, 'e')$. On the other side, the action of user U_2 to delete the last 's' in the word 'systems' is translated on site 2 into the operation $op_2 = \text{delete}(10)$.

Operations op_1 and op_2 are concurrent (they are defined on the same state), but they will be executed by each site serially. Take the case of site 1; if both operations are executed as they are, in the reception order ($op_1; op_2$), the final state will be $O'' = \text{"one systes ..."}$. This state does not correspond to the intention of user U_2 (Fig. 2A). To obtain the result desired by user U_2 , site 1 must not execute op_2 but another operation $op_2' = \text{delete}(11)$ obtained from op_2 by considering that op_1 has been executed before it (Fig. 2B).

Let us interpret the precedent example in the light of what has been said about the user's intention. Users U_1 and U_2 see simultaneously the object in the state $O = \text{"on systems..."}$ and each one has a different intention: $I_1 = \{\text{insert 'e' after 'on'}\}$, $I_2 = \{\text{delete the last 's' of the word 'systems'}\}$. These intentions are realized on the state O by

the operations: $op_1 = \text{Operation}(I_1, O) = \text{insert}(3, 'e')$
 $op_2 = \text{Operation}(I_2, O) = \text{delete}(10).$

On site 1, as op_1 is executed first, the new state of the object is $O' = \text{"one systems..."}$; the intention I_1 is thus realized. Then, when op_2 is executed on the state O' , the resulting new state is $O'' = \text{"one systes..."}$. In this case the intention I_2 is not respected (Fig. 2A). In fact, in order to respect the intention I_2 it would be necessary to execute (Fig. 2B) $op_2' = \text{delete}(11)$ instead of $op_2 = \text{delete}(10)$. We would obtain: $O.op_1.op_2' \rightarrow I_1.I_2$ with:

$op_2' = \text{Operation}(I_2, O.op_1), \quad I_1 = \text{Intention}(op_1, O),$
 $I_2 = \text{Intention}(op_2', O.op_1) = \text{Intention}(op_2, O).$

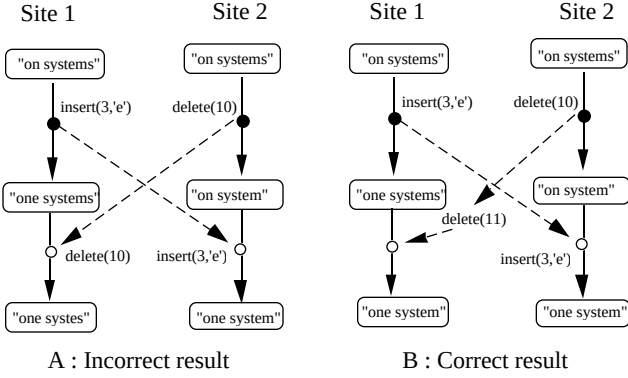


Figure 2. Execution of concurrent operations.

Forward Transposition. In reality, site 1 does not know neither I_1 nor I_2 , which are abstractions specific to the users. However, from concurrent operations op_1 and op_2 (that realize respectively I_1 and I_2), site 1 must determine the operation op_2' to be executed on the object state $O' = O.op_1$ in order to respect the intention I_2 . This determination is possible by defining a function of transposition [4], specific to the application and to the semantic of the operations, that gives for each ordered couple of operations (op_1, op_2) an operation op_2' realizing the same intention as op_2 , while taking the effects of op_1 into account. This function is named *forward transposition* and noted Transpose_fd .

Definition 5. Forward transposition. The forward transposition function, $\text{Transpose_fd}(op_1, op_2)$, delivers as a result an operation noted $op_2^{op_1}$ and named forward transposed operation of op_2 with regard to op_1 , such that:
 $\forall O_i, \text{Intention}(op_2^{op_1}, O_i.op_1) = \text{Intention}(op_2, O_i).$

For example, for the couple (insert, delete), we have :

```

Transpose_fd (insert(p1,c1), insert(p2,c2)) =
  case p1 ? p2 of
    p1 < p2 : return insert(p2 + 1, c2);
    p1 > p2 : return insert(p2, c2);
    p1 = p2 : if c1 = c2 then return id
                else return insert(p2, c2); endif
  endcase;

Transpose_fd (delete(p1), insert(p2,c2)) =
  if p1 < p2 then return insert(p2 - 1, c2)
  else return insert(p2, c2); endif

Transpose_fd (insert(p1,c1), delete(p2)) =
  if p1 ≤ p2 then return delete(p2 + 1)
  else return delete(p2); endif

```

```

Transpose_fd (delete(p1), delete(p2)) =
  case p1 ? p2 of
    p1 < p2 : return delete(p2 - 1);
    p1 > p2 : return delete(p2);
    p1 = p2 : return id
  endcase;

```

In this example, when the operations are identical, the forward transposed operation is the null operation (noted id). This case happens when two users insert the same character, in the same position, at the same moment. One operation must be executed and the other one must be ignored; only one operation is sufficient to satisfy the intention of both users.

4.3 Serialization of an Operation Concurrent to a Sequence of n Operations

Consider the sequence seq_n of n operations (with $seq_n = op_1.op_2.... op_n$) executed in that order on object O from the state O_i on site S . The resulting state is O_f such as $O_i.seq_n = O_f$. Let op be a new operation generated on another site from the state O_i respecting intention I . We say in this case that op has been generated concurrently with the sequence seq_n . As the object state has changed from O_i to O_f , it is no longer possible to execute op on O_i on site S . So, it is necessary to define and execute an operation that realizes I on O_f .

By noting op^{op_1} the forward transposed operation of op with regard to op_1 (i.e. $op^{op_1} = \text{Transpose_fd}(op_1, op)$), $(op^{op_1})^{op_2}$, also denoted $op^{op_1.op_2}$, expresses that op was first transposed forward with op_1 and the resulting operation was then transposed with op_2 . In the same manner the notation op^{seq_n} designates the forward transposed operation of op with the operations of the sequence $seq_n = op_1.op_2.... op_n$.

Definition 6. Forward transposition with a sequence. op^{seq_n} is the resulting operation of a successive forward transposition of op with the operations of the sequence seq_n . It is formally defined by the following recursive formula:

$op^{seq_n} = \text{Transpose_fd}(op_n, op^{seq_{n-1}})$ with:
 $seq_n = op_1.op_2.... op_n = seq_{n-1}.op_n$ and $op^{seq_0} = op$.

The next lemma expresses that the execution of op^{seq_n} from the state O_f respects the intention I of the operation op .

Lemma 3. If op and seq_n operate on the same object state O_i then: $\text{Intention}(op, O_i) = \text{Intention}(op^{seq_n}, O_i.seq_n)$.

4.4 Serialization of n Concurrent Operations

Let $op_1, op_2, ..., op_n$, be n concurrent operations generated on n sites from the same object state O_i and respectively respecting the n intentions $I_1, I_2, ..., I_n$. The problem is to determine the n operations which must be executed serially from O_i to satisfy the series of intentions $I_1.I_2....I_n$.

The solution consists in successively realizing each one of these intentions: op_1 is executed first to realize I_1 (i.e. $O_i.op_1 \rightarrow I_1$), $op_2^{op_1}$ is next executed thus realizing I_2 (i.e. $O_i.op_1.op_2^{op_1} \rightarrow I_1.I_2$), then $op_3^{op_1.op_2^{op_1}}$, the forward transposed operation of op_3 with $op_1.op_2^{op_1}$, is executed to realize I_3 , and so on. More generally, to respect the intention

I_j , we must execute the transposed operation of op_j with regard to the operations that have been executed to realize the series of intentions $I_1.I_2....I_{j-1}$. Finally, the operation sequence that respects the n intentions is the following:

$$(op_1).(op_2^{op_1}).(op_3^{(op_1).(op_2^{op_1})})..... \\(op_n^{(op_1).(op_2^{op_1}).....(op_{n-1}^{op_1}....)}).$$

To simplify the previous formula, we will use the notation $op_1:op_2:op_3 :...:op_n$, in which the operator ':' preceding an operation signifies that this operation is transposed forward with the operation sequence in the left side of the operator. Formally, this means:

$seq:op = seq.op^{seq}$ where seq is a sequence.

Lemma 4. Let n concurrent operations $op_1, op_2, ..., op_n$ such as: $(O_i.op_1 \rightarrow I_1), (O_i.op_2 \rightarrow I_2), ..., (O_i.op_n \rightarrow I_n)$. We have: $O_i[op_1:op_2: ...:op_n] \rightarrow I_1.I_2....I_n$. The proof of this lemma can be found in [17]

4.5 Serialization of a Sequence Concurrent to an Operation

Partial Concurrency Problem. Let op_1, op_2 be two concurrent operations, such as op_1 is generated from the state O_1 on site 1, and op_2 from the state O_2 (with $O_1 \neq O_2$), on site 2. As op_1 and op_2 are not generated from the same state, they are said to be *partially concurrent* [1]. In this case, application of the forward transposition may lead to a wrong result. The following example illustrates a partial concurrency situation.

Example 2. Consider two users who try to change the word "telephone" into "telephone" as in (Fig. 3). We observe that op_1 precedes op_2 , and op_3 is concurrent to op_1 , but op_3 is partially concurrent to op_2 insofar as op_2 and op_3 are not generated from the same state.

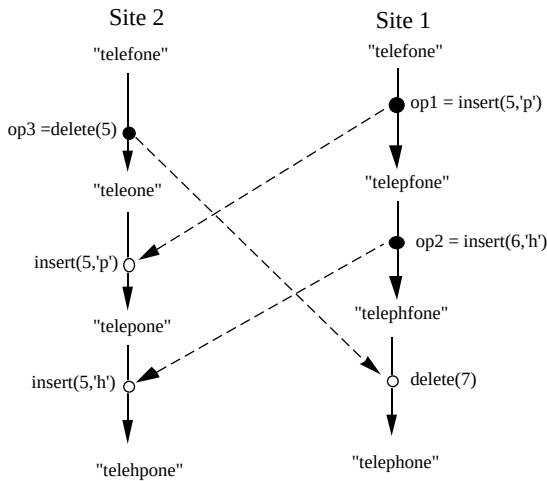


Figure 3. Partial concurrency between operations.

On site 1, op_3 has to be transposed with the sequence $op_1.op_2$; it is first transposed forward with op_1 then with op_2 . The execution of the resulting operation $op_3'' = delete(7)$ gives the wanted word "telephone". The problem comes from site 2: the transposition of op_1 with op_3 gives $op_1' = op_1 = insert(5,'p')$, and the transposition of op_2 with op_3 gives $op_2' = insert(5,'h')$ whose execution leads to

"telephone" which is different from what is obtained on site 1.

This result, which does not correspond to the user intention, is due to a wrong application of the forward transposition function to the operations op_2 and op_3 on site 2. This function requires that both operations are defined on the same state. This is not the case on site 2 where $delete(5)$ is generated from the state "telephone" while $insert(6,'h')$ is generated on site 1 from the word "telephone". For that reason the forward transposition of $insert(6,'h')$ with $delete(5)$ gives a wrong result. In order to solve this partial concurrency problem, the history must be re-ordered without violating the intentions of the users so that the forward transposition can be applied correctly. To do this, we will use another type of transposition called *backward transposition*. This transposition was introduced in [14] to undo an operation in a history without undoing the operations that follow it.

Backward Transposition. Suppose that on a site S , the history $op_1.op_2$ realizes $I_1.I_2$. Changing the execution order of op_1 and op_2 means finding the history $op_2'.op_1'$ that would have existed if op_2 had been executed before op_1 . This history would have realized $I_2.I_1$. We define a function that enables the execution order between two operations to be exchanged without violating their intentions. This function is called *backward transposition*, noted $Transpose_bk$; it enables an operation to be moved back to the past and to simulate it being executed first.

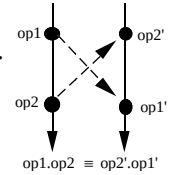
Definition 7 Backward transposition.

Let $op_1.op_2$ be a sequence of two operations.

$Transpose_bk(op_1, op_2) = op_2'.op_1'$ with:

$op_2' = Transpose_fd(op_1, op_2')$ and

$op_1' = Transpose_fd(op_2', op_1)$.



$Transpose_bk$ is specified as $Transpose_fd$ for every couple of operations $(op_1.op_2)$. For example:

$Transpose_bk(insert(p_1,c), delete(p_2)) =$

case $p_1 ? p_2$ *of*

$p_1 < p_2$: *return* $(delete(p_2 - 1), insert(p_1,c));$

$p_1 > p_2$: *return* $(delete(p_2), insert(p_1 - 1,c));$

$p_1 = p_2$: *return* $(id, id);$

endcase

5 Maintaining the Consistency of the Copies

5.1 Case of Two Concurrent Operations

Consider two concurrent operations op_1 and op_2 , respectively generated by site 1 and site 2 from the state x (Fig. 4). When site 1 receives op_2 , it cannot execute this operation on the state x because op_1 has changed the object state to y . So, op_2 must be transposed forward with op_1 so as to obtain an operation defined on y that respects the intention of the user of site 2. It is the same for op_1 which must be transposed forward with op_2 on site 2.

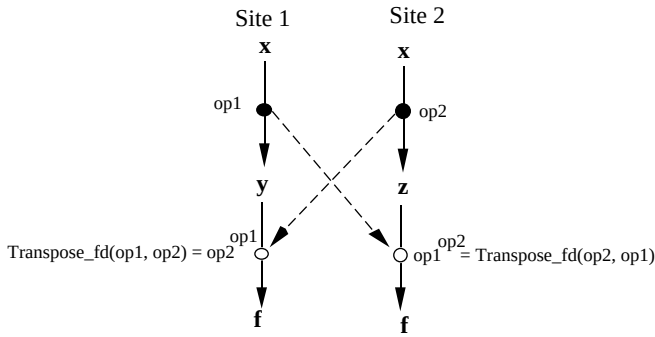


Figure 4. Forward Transposition.

In order to keep both object copies consistent, it is necessary, in this example, that $x.op_1.op_2^{op_1}$ be equal to $x.op_2.op_1^{op_2}$. More generally, to guarantee the consistency of the copies, the forward transposition must verify the following condition.

Condition 1. Let op_1 and op_2 be two operations such as:
 $op_2^{op_1} = \text{Transpose_fd}(op_1, op_2)$ and
 $op_1^{op_2} = \text{Transpose_fd}(op_2, op_1)$
 then $op_1.op_2^{op_1} \equiv op_2.op_1^{op_2}$.

As $op_1.op_2^{op_1} = op_1.op_2$ and $op_2.op_1^{op_2} = op_2.op_1$, condition 1 can also be written under the following form: $op_1.op_2 \equiv op_2.op_1$.

It results that, as Transpose_fd verifies condition 1, Transpose_bk verifies the following property:
 $\text{Transpose_bk}(op_1.op_2) = op_2'.op_1' \Rightarrow op_1.op_2 \equiv op_2'.op_1'$.

The forward transposition functions for the operations insert and delete previously defined, verify condition 1 except when two insertion operations insert two different characters at the same position. In this case, a choice has to be made: which character must be inserted before the other? The solution that we have adopted consists in associating a priority to each character (for example, the character's code). In case of a conflict, the character whose priority $pr(c)$ is the highest, is inserted before the other. Taking this modification into account, the definition of Transpose_fd for these two operations becomes:

$\text{Transpose_fd}(\text{insert}(p_1, c_1), \text{insert}(p_2, c_2)) =$

case $p_1 ? p_2$ of

$p_1 < p_2$: return $\text{insert}(p_2 + 1, c_2)$;

$p_1 > p_2$: return $\text{insert}(p_2, c_2)$;

$p_1 = p_2$: if $c_1 = c_2$ then return id

elseif $pr(c_2) > pr(c_1)$ then return $\text{insert}(p_2, c_2)$
else return $\text{insert}(p_2 + 1, c_2)$; endif;

endif;

endcase

The part inside the frame contains the modifications added to the definition given in the paragraph 4.2.

5.2 Generalization to n Concurrent Operations

Suppose now that n sites generate, from the same state O_i of the object O , n concurrent operations $op_1, op_2, \dots, op_n$ realizing respectively the n intentions $I_1, I_2, \dots, I_n$. These operations are broadcast to all sites. They may be received in any order. If they arrive at a site S in the order $op_1, op_2, \dots, op_n$ then the sequence executed on this site will be $(op_1:op_2:$

$\dots:op_n)$. This sequence respects the series of intentions $I_1, I_2, \dots, I_n$. The problem is then to satisfy the consistency property, so as to guarantee the consistency of the copies.

Consistency Property. Let $(op_1, op_2, \dots, op_n)$ be n operations generated in parallel and respecting the intentions $(I_1, I_2, \dots, I_n)$. Let f be an integer function $[1-n] \rightarrow [1-n]$.

The consistency property expresses that:

the realization of $[I_{f(1)}.I_{f(2)}.\dots.I_{f(n)}]$ is equivalent to the realization of $[I_1.I_2.\dots.I_n]$. In other words:

$$op_{f(1)}:op_{f(2)}:\dots:op_{f(n)} \equiv op_1:op_2:\dots:op_n.$$

As seen previously, condition 1 is necessary to guarantee that the realization of $I_1.I_2$ is equivalent to that of $I_2.I_1$. The next example shows that this condition is not sufficient to prove the consistency property in the general case of n operations.

Example 3. Suppose that three users on three sites see the word "buver" and they want to change it to "buffer". So:

1. the first user adds 'f' after the 'u': $op_1 = \text{insert}(3, 'f')$;
2. the second user deletes the 'v': $op_2 = \text{delete}(3)$;
3. the third user adds 'f' after the 'v': $op_3 = \text{insert}(4, 'f')$.

These operations are broadcast to all sites, then received and executed after forward transposition as shown in Fig. 5.

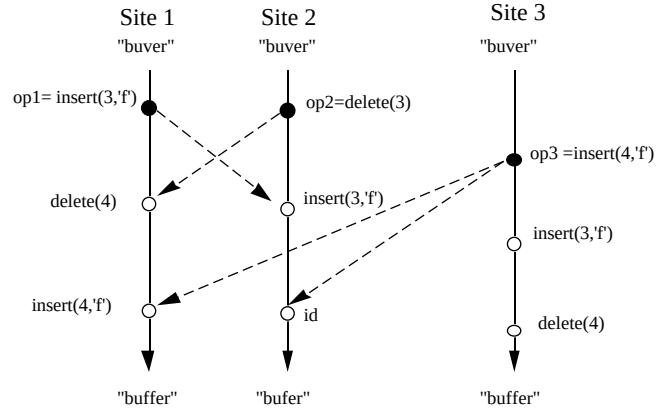


Figure 5. Consistence property is not respected.

On site 1, the local operation $\text{insert}(3, 'f')$ is executed. Then $\text{delete}(3)$ is received; it is transposed forward with $\text{insert}(3, 'f')$ and becomes $\text{delete}(4)$. When the operation $\text{insert}(4, 'f')$ arrives, it is transposed forward with $\text{insert}(3, 'f')$, then with $\text{delete}(4)$, so it becomes $\text{insert}(4, 'f')$. The word finally obtained is "buffer".

On site 2, the local operation $\text{delete}(3)$ is executed, then $\text{insert}(3, 'f')$ is received. It is not modified by the forward transposition with $\text{delete}(3)$. The last received operation $\text{insert}(4, 'f')$ is transposed forward with $\text{delete}(3)$ and becomes $\text{insert}(3, 'f')$; it is then transposed forward with $\text{insert}(3, 'f')$ which gives the nul operation (id). The word finally obtained is "bufer" which is different from that obtained on site 1.

So, the objective is now to determine a sufficient condition to be verified by the function Transpose_fd in order to guarantee the consistency property.

When analysing the previous example, we notice that the histories on site 1 and site 2 are not equivalent. To obtain the equivalence, the transposition of the third

operation $\text{insert}(4, 'f')$ should be the same on site1 and on site 2. In other words, the forward transposition must give:
 $\text{insert}(4, 'f')\text{insert}(3, 'f')\text{delete}(3) = \text{insert}(4, 'f')\text{delete}(3)\text{insert}(3, 'f')$.

This means that the forward transposition of one operation with a sequence of two operations does not depend on the order in which the operations of the sequence are transposed. This constraint is expressed by Condition 2.

Condition 2. Whatever the operations op_i , op_j and op_k be, Transpose_fd must verify:
 $op_k^{op_i:op_j} = op_k^{op_j:op_i}$.

We have proved that conditions 1 and 2 are sufficient to guarantee the consistency property. We have expressed these results by the following theorem. The proof will not be presented here due to lack of space.

Consistency Theorem. If whatever the concurrent operations op_i , op_j and op_k are, the function Transpose_fd verifies the following conditions:

Condition 1: $[op_i:op_j] \equiv [op_j:op_i]$,

Condition 2: $op_k^{op_i:op_j} = op_k^{op_j:op_i}$,

then: $\forall n$ concurrent operations $op_1, op_2, \dots, op_n$ and

$\forall f$ an integer functions $[1-n] \rightarrow [1-n]$:

$op_{f(1)}:op_{f(2)}: \dots:op_{f(n)} \equiv op_1:op_2: \dots:op_n$

After studying and evaluating both possibilities $op_k^{op_i:op_j}$ and $op_k^{op_j:op_i}$ for every triplet of operations $(op_i, op_j, op_k) \in \{\text{insert}, \text{delete}\}^3$, it appears that the problem comes from the conflict between two insertion operations of a character at the same position. To solve this kind of conflict, we need to know how these operations were generated. Two cases are possible: either two users have inserted two characters at the same position, or two users have inserted the two characters at two different positions and a third user has concurrently deleted characters between these positions.

In order to determine the correct case, two additional parameters (b and a) are added to each insertion operation. b , (resp. a) is the set of the operations that have deleted a character *before* (resp. *after*) the inserted character. So, for two operations $\text{insert}(p, c_1, b_1, a_1)$ and $\text{insert}(p, c_2, b_2, a_2)$, defined on the same state, we have one of the following cases:

- $b_1 \cap a_2 \neq \emptyset \Rightarrow c_2$ was inserted before c_1 .

- $a_1 \cap b_2 \neq \emptyset \Rightarrow c_2$ was inserted after c_1 .

- $b_1 \cap a_2 = a_1 \cap b_2 = \emptyset \Rightarrow c_1$ and c_2 were inserted at the same position.

So, Transpose_fd must be modified as follows:

$\text{Transpose_fd}(\text{insert}(p_1, c_1, b_1, a_1), \text{insert}(p_2, c_2, b_2, a_2)) =$
 $\text{case } p_1 ? p_2 \text{ of}$

$p_1 < p_2 : \text{return } \text{insert}(p_2 + 1, c_2, b_2, a_2);$

$p_1 > p_2 : \text{return } \text{insert}(p_2, c_2, b_2, a_2);$

$p_1 = p_2 :$

$\text{if } (a_1 \cap b_2 \neq \emptyset) \text{ then return}(\text{insert}(p_2 + 1, c_2, b_2, a_2))$

$\text{elseif } (b_1 \cap a_2 \neq \emptyset) \text{ then return}(\text{insert}(p_2, c_2, b_2, a_2))$

$\text{elseif } pr(c_2) > pr(c_1) \text{ then return } \text{insert}(p_2, c_2)$

$\text{elseif } pr(c_2) < pr(c_1) \text{ then return } \text{insert}(p_2 + 1, c_2)$

$\text{else return id();endif;}$

endcase.

$\text{Transpose_fd}(\text{delete}(p_1), \text{insert}(p_2, c_2, b, a)) =$
 $\text{if } p_1 < p_2 \text{ then return } \text{insert}(p_2 - 1, c_2, b + \text{delete}(p_1), a)$
 $\text{else return } \text{insert}(p_2, c_2, b, a + \text{delete}(p_1)); \text{endif}$

6 Integration of an Operation in the History

6.1 Problematic

Let $H_S(n) = op_1.op_2....op_n$ be the history of the object O on the site S . O_i (resp. O_f) is the initial (resp. current) state of that object. Let op be an operation sent by S' and received by S . This operation was generated from the state O_m in order to realize the intention $I = \text{Intention}(op, O_m)$. To integrate op in $H_S(n)$ means determining and executing on the current state O_f an operation op_{n+1} that respects I , that is $\text{Intention}(op_{n+1}, O_f) = \text{Intention}(op, O_m) = I$. Due to the reception procedure, the history $H_S(n)$ verifies:

- all operations that precede op are integrated in $H_S(n)$;
- op does not precede any operation of $H_S(n)$.

The causality is thus respected, and it remains to transpose op forward with its concurrent operations to obtain an operation whose execution respects the user intention and guarantees the data consistency. To achieve this, the naive method consists of going through the history $H_S(n)$ and transposing op forward with each operation concurrent to it [4]. This method is implemented by the following algorithm.

Notations:

op : received operation

S_{op} : op 's generator site

V_{op} : state vector associated to op

H : representation of $H_S(n) = \text{array of } n \text{ elements of the form } \langle op_i, S_{opi}, V_{opi} \rangle$

n : number of elements in H .

procedure Integrate ($\langle op, S_{op}, V_{op} \rangle$):

(for $i := 1$ to n do

$\langle op_i, S_{opi}, V_{opi} \rangle := H[i];$

if $V_{opi}[S_{opi}] \geq V_{op}[S_{opi}]$ then

{ op_i // op according to lemma 2 }

$op := \text{Transpose_fd}(op_i, op); \text{endif;}$

endfor;

execute (op);

$H[n+1] := \langle op, S_{op}, V_{op} \rangle;$

$n := n + 1;$

The previous algorithm produces in some cases inconsistent results characterised by the divergence of the copies. These cases correspond to a partial concurrency situation which is not treated by this algorithm.

In order to solve this problem, the history must be reordered so that the forward transposition can be applied correctly. Let $H_S(m)$ be the history of the site S' where op was generated, taken at that instant. If O_i was the initial state, then op was generated from the state: $O_i.H_S(m) = O_m$. Notice that all the operations of $H_S(m)$ precede op and that all these operations were already in $H_S(n)$. This gives us the idea to reorder $H_S(n)$ in a way that its operations are regrouped into two sequences seq_1 and seq_2 such as:

$H_S(n) \equiv \text{seq}_1.\text{seq}_2$ with:

$\text{seq}_1 \equiv H_S(m)$ = the sequence of operations that precede op ,

seq_2 = the sequence of operations concurrent to op .

Under these conditions, seq_2 is defined on the state $O_i.\text{seq}_1 = O_i.H_S(m) = O_m$. Consequently, seq_2 and op are defined on the same state. This enables lemma 3 to be applied to obtain: $\text{Intention}(op, O_m) = \text{Intention}(op^{\text{seq}_2}, O_m.\text{seq}_2)$

which gives: $H_S(n+1) \equiv \text{seq}_1.\text{seq}_2.op^{\text{seq}_2}$.

The problem that now needs to be solved is the following: how to reorder $H_S(n)$ into two sequences without violating the intentions of the users.

6.2 Reordering $H_S(n)$ into Two Sequences

Let $H_S(n) = op_1.op_2....op_i.op_{i+1}....op_n$ be a history that respects the intentions $I_1.I_2...I_i.I_{i+1}....I_n$. Let $op_{i+1}.op_i' = \text{Transpose_bk}(op_i, op_{i+1})$. Since $op_{i+1}.op_i' \equiv op_i.op_{i+1}$, the execution order of op_i and op_{i+1} in $H_S(n)$ could be changed without touching the other operations or violating their intentions; this gives $H_S(n) \equiv op_1.op_2...op_{i+1}.op_i'...op_n$ that respects the intentions $I_1.I_2....I_{i+1}.I_i....I_n$.

Every application of the backward transposition enables two operations to be permuted. It is thus possible to reorder the history in any order. To do this, it is sufficient to know the necessary permutations, then to apply the transpositions that correspond to these permutations. For our problem, $H_S(n)$ must be reordered so that the operations which precede op , the operation to be integrated, are regrouped before the operations which are concurrent to op . In order to do this, Transpose_bk must be applied to each couple (op_i, op_j) so that op_j precedes op_i , and op_i is concurrent to op . This must be repeated until all the operations that precede op are located before the operations which are concurrent to op . In this way, $H_S(n)$ will be separated into two sequences seq_1 and seq_2 , so that all the operations of seq_1 precede op and all the operations of seq_2 are concurrent to op .

6.3 Algorithm

The procedure $\text{Transpose_backward}$ redefines the function Transpose_bk so that it transposes backward two operations in the history $H_S(n)$. The function Separate separates the history $H_S(n)$ into two sequences seq_1 and seq_2 , and returns the length n_1 of seq_1 . The procedure Integrate transposes forward the operation op with the operations of seq_2 then it executes the resulting operation. This procedure is called by the reception procedure presented in 3.3.

Notations:

op : received operation

S_{op} : op 's generator site

V_{op} : state vector associated to op

H : representation of $H_S(n)$ = array of n elements of the form $\langle op_i, S_{op_i}, V_{op_i} \rangle$

n : number of elements in H

n_1 : length of seq_1 calculated by the procedure Separate .

procedure $\text{Transpose_backward}(j)$: {Transpose backward the j^{th} operation with the $(j-1)^{\text{th}}$ operation}

($\langle op_j, S_{op_j}, V_{op_j} \rangle := H[j]$;

$\langle op_{j-1}, S_{op_{j-1}}, V_{op_{j-1}} \rangle := H[j-1]$;

$op_j.op_{j-1} = \text{Transpose_bk}(op_{j-1}, op_j)$;

$H[j] := \langle op_{j-1}, S_{op_{j-1}}, V_{op_{j-1}} \rangle$;

$H[j-1] := \langle op_j, S_{op_j}, V_{op_j} \rangle$;

function Separate ($\langle op, S_{op}, V_{op} \rangle$) :

($n_1 := 0$;

for $i := 1$ **up to** n **do**

$\langle op_i, S_{op_i}, V_{op_i} \rangle := H[i]$;

if $V_{op_i}[S_{op_i}] < V_{op}[S_{op_i}]$ **then** { op_i precedes op }

for $j := i$ **down to** $n_1 + 2$ **do**

{transpose op_i backward to seq_1 }

$\text{Transpose_bk}(j)$; **endfor**;

$n_1 := n_1 + 1$; **endif**; **endfor**;

return (n_1);)

procedure Integrate ($\langle op, S_{op}, V_{op} \rangle$) :

($n_1 := \text{Separate}(\langle op, S_{op}, V_{op} \rangle)$;

for $i := n_1 + 1$ **up to** n **do** {transpose op forward with regard to seq_2 }

$\langle op_i, S_{op_i}, V_{op_i} \rangle := H[i]$;

$op := \text{Transpose_fd}(op_i, op)$; **endfor**;

execute (op);

$n := n + 1$;

$H[n] := \langle op, S_{op}, V_{op} \rangle$;

Let us see an illustration of this algorithm in Example 2.

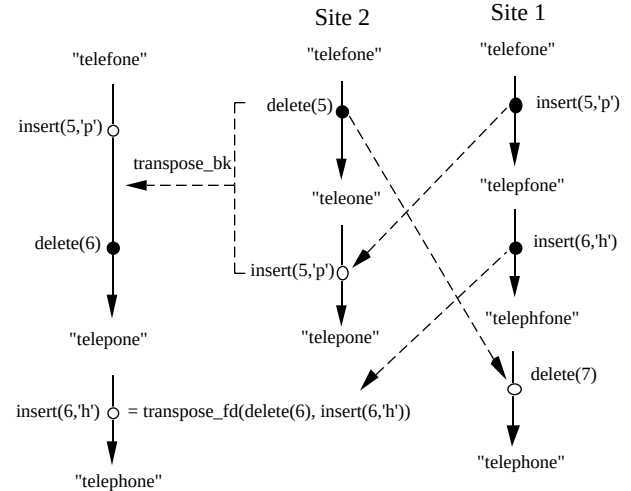


Figure 6. Solution for the partial concurrency.

The purpose is to integrate the operation $\text{insert}(6, 'h')$ in the history of site 2: $H(n) = \text{delete}(5).\text{insert}(5, 'p')$. In this history (Fig. 6), $\text{insert}(5, 'p')$ is the only operation that precedes the operation $\text{insert}(6, 'h')$. So, to separate $H(n)$ into two sequences seq_1 and seq_2 , both operations of $H(n)$ must be transposed backward. This gives:

$H(n) = \text{Transpose_bk}(\text{delete}(5), \text{insert}(5, 'p')) = \text{insert}(5, 'p').\text{delete}(6),$

$\text{seq}_1 = \text{insert}(5, 'p')$ and $\text{seq}_2 = \text{delete}(6)$.

It is clear that $\text{delete}(6)$ and $\text{insert}(6, 'h')$ are now defined on the same state, (corresponding to the word "telephone" obtained after the execution of $\text{insert}(5, 'p')$ from the word "telephone"). When transposing forward $\text{insert}(6, 'h')$ with $\text{delete}(6)$, the results is the operation $\text{insert}(6, 'h')$ to be executed on the word " telephone". Thus, the final word obtained is "telephone".

7 Supporting Mobile Sites

In this section, we assume that some sites of the distributed system are mobile ones, and we investigate the possibility for mobile site to be temporarily disconnected while the user at this site continues working on the local object's copy. So, we distinguish two types of collaborations: on-line collaboration and off-line collaboration.

The on-line collaboration is exactly the same as in the distributed model seen before; sites are connected to each other, and they exchange operations by using the same algorithm in order to guarantee consistency of the object. In off-line collaboration, user works on an autonomous site (disconnected from the other sites) and modifies only the local object's copy. So this copy will be different from the copies on the other sites until the site is reconnected and the modifications are integrated on all sites. However, during off-line collaboration, the use of the state vectors remains the same. So, when a site receives and executes an operation from another site, it increments the corresponding element of its state vector. Consequently, as the disconnected site does not receive any operation from other sites, the only element that evolves in its state vector is that corresponding to the site itself.

Now, all that we need to precise is, how to pass from one type of collaboration to the another one. To achieve this, we will define the procedures for disconnecting/reconnecting a site.

7.1 Disconnection of a Site

When a user wants to pass from on-line collaboration to off-line collaboration, he/she executes the procedure *Disconnect*. This procedure broadcasts a message to all sites in order to announce the disconnection. Then, it disables broadcast and reception of operations and it defines a pointer (*begin_local_history*) to locate the part of history that will be added in the off-line collaboration.

procedure Disconnect {*procedure executed on site S*}

(broadcast (<disconnection, S>);
 disable broadcast;
 disable reception;
begin_local_history := *n*); {*n* is the number of elements in the history H_S on the site S}

Finally, we suppose that each site manages an OLS set containing the on-line sites. So, when a site receives a disconnection announcement, all that this site must do, is to remove the disconnect site from its OLS set.

7.2 Reconnection of a Site

Reconnection is more complicated than disconnection. Modifications made during off-line collaboration must be integrated in order to obtain the same copies of the object on all sites; and the histories must be merged to make all histories equivalent. So, in order to reconnect a site S to on-line collaboration session, the following steps are needed :

(1) At first, the site S updates its OLS set after having obtained it from another on-line site.

(2) In order to integrate the operations that have been generated and executed on the site S into the histories of the

other sites, S broadcasts to all sites in OLS set the message <reconnect, S, seq, n_{seq} >, where seq is the sequence of the operations executed by S during the off-line session, and n_{seq} is the number of operations in seq.

(3) When a site receives a message <reconnect, S, seq, n_{seq} >, it merges the operations of seq into its history. The merging is done by applying to each operation of seq, the procedure *Integrate*, seen before.

(4) The last step enables integrating into the history of S, the operations generated and executed on the other sites. There are two possible strategies. In the first one, the site S recovers the operations executed on the other sites during disconnection and it integrates them into its history. In the second strategy, S recovers, from an on-line site S_v , a current copy of the object and its current history. We will detail only the second strategy that corresponds to the simplest solution.

S designates a site S_v as a server for the reconnection in the reconnection message. This site will send to S its current copy of the object with its current history. Before doing that, S_v must be sure that there is no operation generated by a site S_i that has not yet been received by it. For that reason, each site S_i in OLS set (with $S_i \neq S_v$) sends an acknowledgement to S_v when it receives the reconnection message from S. The site S_v waits until it receives the acknowledgement messages from all the sites, then it sends the copy of the object with its history. During the reconnection phase, the server S_v can always execute operations received from any site S_i as long as it has not received the acknowledgement from this site. On the other hand, execution of the operations sent by S_i after its acknowledgement message, will be delayed on S_v until the end of the reconnection phase.

Notations:

S_i, S_j : *reconnected site, a site of OLS;*
 S_v : *selected site to be the reconnection server site;*
 O, O_{S_i} : *object copy on S (respec. S_i)*
 H_S, H_{S_i} : *history of S (respec. S_i) = arrays of elements of the form < op_i, S_{opi}, V_{opi} >;*
 n, n_{S_i} : *number of operations in H_S and H_{S_i} ;*
 seq : *sequence of operations generated on S during off-line session;*
 n_{seq} : *number of operations in seq;*

procedure Reconnect_client () ;

(enable broadcast;
 enable reception;
 update OLS;
 S_v := *selected site of OLS;*
 n_{seq} := n - *begin_local_history;*
 for $i := 1$ up to n_{seq} do {construction of seq}
 seq[i] := $H_S[i + \text{begin_local_history}]$; endfor;
 broadcast (<reconnect, S, seq, n_{seq}, S_v >);
 wait until reception < $O_{S_i}, H_{S_i}, n_{S_i}$ >;
 $n = n_{S_i}; H_S := H_{S_i}; O := O_{S_i};$)

The next procedure is called on each site S_i after reception of the message <reconnect, S, seq, n_{seq}, S_v >.

```

procedure Reconnect_server (<reconnect, S, seq, nseq, Sv>);
  ( if Si ≠ Sv then send(Sv, ack) {send an acknowledgement to Sv}
  else wait until reception <ackj> from Sj (∀Sj ∈ (OLSSv - {Sv})
endif;
  for i:=1 up to nseq do {integration of seq}
    Integrate (seq[i]); endfor;
  if Si = Sv then {this site is the server site}
    send(S, <OSi, HSi, nSi>); {send the object and its history to S}
  endif;
  OLSSi := OLSSi + {S}; )

```

8 Conclusion

We propose a new approach for concurrency control in collaborative systems. This approach enables the history associated with an object to be constructed incrementally, without ever undoing and redoing old operations. It is based on the notion of intention, that enables equivalent histories to be characterised. Formally, this equivalence is obtained by semantic properties of operations that we call forward transposition and backward transposition. Forward transposition resolves the problem of concurrent operations, whilst backward transposition changes the order of operations in the history while preserving equivalence.

Implementation in a distributed environment is proposed. Finally, we have shown that the distributed model is very well adapted to mobile systems in which the reconnection of a site requires old operations to be integrated into the history of each site.

9 References

- [1] Allison C.; "Concurrency Control for Real Time Groupware," CE94: Concurrent Engineering Research and Applications, A global Perspective, Pittsburg, August 1994, pp. 163-170.
- [2] Cart M., Ferrié J., Richy H.; "Contrôle de l'exécution de transactions concurrentes," Technique et Science Informatiques, vol. 8, n°. 3, pp. 225-240, 1989.
- [3] Decouchant D., Quint V., Vatton I.; "L'édition Coopérative de documents avec Griffon," Colloque IHM'92, Paris, Décembre 1992.
- [4] Ellis C.A., Gibbs S.J.; "Concurency Control in Groupware Systems," in Proc. ACM Int. Conf. on Management of Data (SIGMOD'89), Seattle, May 1989, pp. 399-407.
- [5] Ellis C.A., Gibbs S.J., Rein G.L.; "Groupware: Some issues and experiences," Commun. ACM, vol. 34, n°. 1, pp. 39-59, January 1991.
- [6] Fish R.S., Kraut R.E., Leland M.D.P., Cohen M.; "Quilt: a collaborative tool for cooperative writing," in Proc. ACM Int. Conf. on Office Information Systems, Palo Alto, March 1988, pp. 30-37.
- [7] Greenberg S., Roseman M., Webster D., Bohnet R.; "Issues and Experiences Designing and Implementing Tow Group Drawing Tools," in Proc. 25th Annual Hawaii Int. Conf. on System Sciences, 1992, vol. 4, pp. 139-150.
- [8] Greenberg S., Marwood D.; "Real Time Groupware as a Distributed System: Concurrency Control and its Effect on the Interface," in Proc. ACM Int. Conf. on Computer Supported Cooperative Work, Canada, October 1994, pp. 207-217.
- [9] Greif I., Seliger R., Weihi W.; "Atomic Data Abstraction in a Distributed Collaborative Editing System," in Proc. 13th ACM Annual Symposium on Principles of Programming Languages, Petersburg, January 1986, pp. 160-172.
- [10] Karsenty A., Beaudouin-Lafon M.; "An Algorithm for Distributed Groupware Applications," in Proc. 13th Int. Conf. on Distributed Computing Systems (ICDCS'93), Pittsburgh, May 1993, pp.195-202.
- [11] Kanawati R.; "Replicated Data Management Algorithm for Distributed Synchronous Groupware Applications," in Proc. DAPSYS'96 'Environnements and Tools', Miskloc (Hungary), October 1996.
- [12] Lamport L.; "Time, Clocks, and the Ordering of Events in Distributed System," Commun. ACM, vol. 21, n°. 7, pp. 558-565, July 1978.
- [13] Mattern F.; "Virtual time and global states of Distributed Systems," in Proc. Int. Workshop on Parallel and Distributed Algorithms, Elsevier Pub., 1989, pp. 215-276.
- [14] Prakash A., Knister M.J.; "Undoing Actions in Collaborative Work," in Proc. ACM Int. Conf. on Computer Supported Cooperative Work, November 1992, pp. 273-280.
- [15] Prakash A., Shim H. S.; "DistView: Support for Building Efficient Collaborative Applications using Replicated Object," in Proc. ACM Int. Conf. on Computer Supported Cooperative Work, October 1994, pp. 153-164.
- [16] Raynal M.; "La communication et le temps dans les réseaux et les systèmes répartis," Eyrolles Pub., Collection DER-EDF, 1991.
- [17] Suleiman M., Cart M., Ferrié J.; "Serialization of Concurrent Operations in a Distributed Collaborative Environment," in Proc. ACM Int. Conf. on Supporting Group Work, November 1997.
- [18] Sun C., Yang Y., Zhang Y., Chen D.; "Distributed Concurrency Control in Real-time Cooperative Editing Systems," in Proc. 2nd Asian Computing Science Conf. on Concurrency and Parallelism, Programming, Networking, Singapour, December 1996, pp.84-95.
- [19] Weihi W.E.; "Commutativity-Based Concurrency Control for Abstract Data Type," IEEE Transactions on Computers, vol. 37, n°. 12, pp. 1488-1505, December 1988.